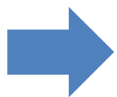


- 要求：了解机器学习的基本概念，掌握线性规划、逻辑回归、支持向量机、决策树、**K-means**聚类、**PCA**降维等常用机器学习方法及应用，了解机器学习模型训练的常用技巧。
- 重点：常用机器学习方法
- 难点：逻辑回归、支持向量机、决策树



机器学习

学习内容 CONTENTS



1

机器学习概述

2

线性回归与逻辑回归

3

支持向量机

4

聚类与降维

5

决策树

6

强化学习

- » 1959 年7 月，IBM 公司的工程师阿瑟·塞缪尔 (Arthur Samuel) 第一次使用了“机器学习 (machine learning)”
- » 定义：让机器具有不需要明确编程而具有的一种学习能力 (the ability to learn without being explicitly programmed)
- » 目标：构造一种学习机器 (learning machine)，使之像人一样具有自我学习能力，而非按部就班完成预设任务

Samuel, Some Studies in Machine Learning Using the Game of Checkers, 1959



Arthur Samuel
1901 – 1990



机器学习



历史数据

训练

新的数据

输入

模型

预测

未知属性

人类



经验

归纳

新的问题

输入

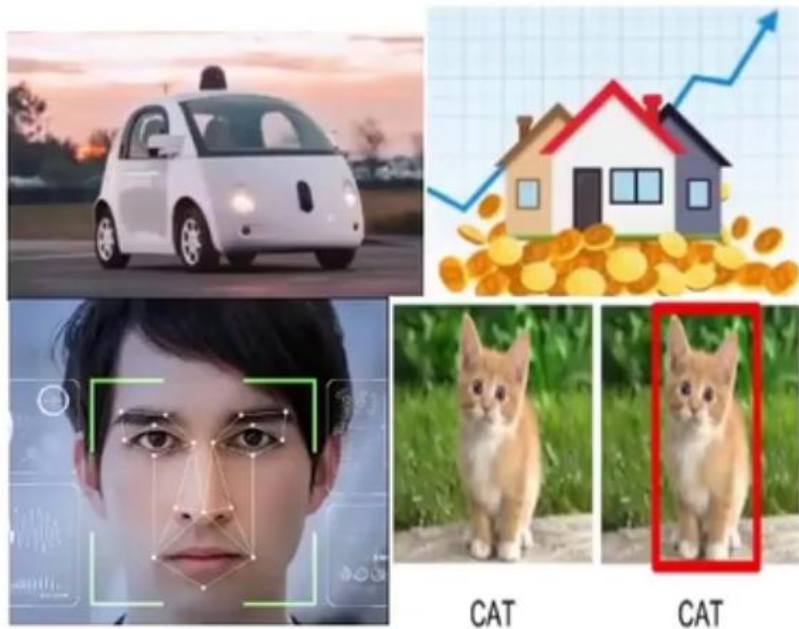
规律

预测

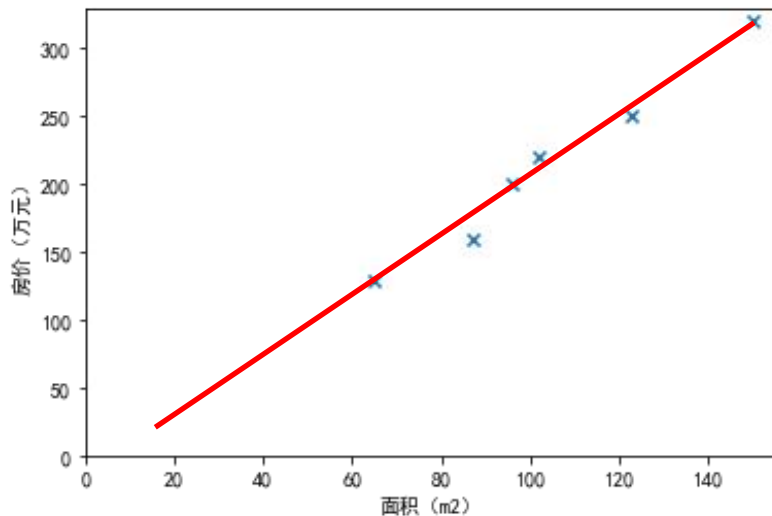
未来

机器学习从经验（数据）中自动分析获得规律，并利用规律对新数据进行预测。

机器学习基于**数据(经验)**，针对具体问题**构建数学模型**，实现对已有数据的准确解析或对未来数据的准确**分类、识别和预测**。



$$\text{房价} = \text{面积} * a + b$$



房价预测问题

从数据中学习实现自我优化与升级



Training examples showing the function $f(x)$ mapping input images (x) to output labels (y):

- $f(\text{brown dog}) = \text{狗}$
- $f(\text{white dog}) = \text{狗}$
- $f(\text{Siamese cat}) = \text{猫}$
- $f(\text{white dog}) = \text{狗}$

Unknown test data examples showing the function $f(x)$ mapping input images (x) to unknown output labels (y):

- $f(\text{German Shepherd}) = ?$
- $f(\text{Persian cat}) = ?$

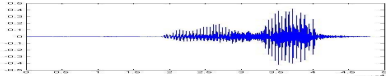
将训练数据喂给计算机，计算机自动求解数据关系，在新的数据上做出预测或给出建议

有监督的机器学习 $\approx$ 构建一个映射函数（模型）： $y = f(x)$

► 图像识别

$f(\text{) = \text{“狗”}$

► 语音识别

$f(\text{) = \text{“同学，你好”}$

► 对话系统

$f(\text{“今天天气怎么样”}) = \text{“杭州今天天气是晴”}$

► 围棋游戏

$f(\text{) = \text{“5-5” (落子位置)}$

用户输入数据 x

机器输出结果 y

监督学习 (Supervised learning) : 学习带有标签的原始数据, 目标是发现原始数据与标签之间的映射关系, 从而预测新数据。

● **回归** (Regression、Prediction)

标签
连续

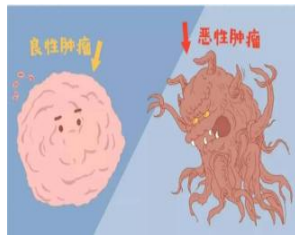
- ✓ 如何预测杭州滨江的房价?
- ✓ 未来的中国股票市场走向?



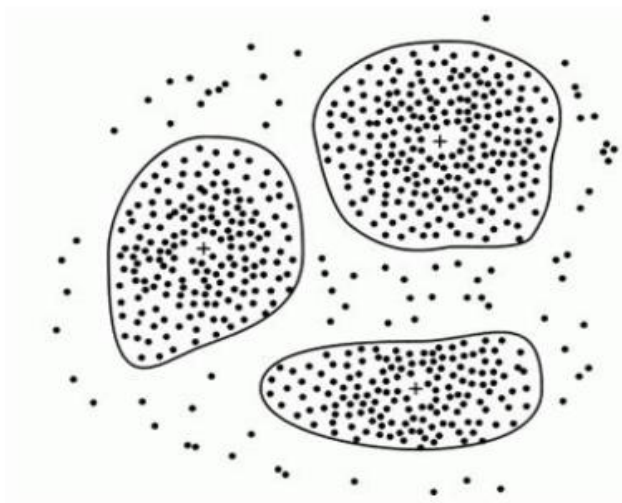
● **分类** (Classification)

标签
离散

- ✓ 根据肿瘤的体积、患者的年龄来判断良性或恶性?
- ✓ 身高1.85m, 体重92kg的男人穿什么尺码的T恤?



无监督学习 (unsupervised learning) : **没有标注的训练数据集** , 需要根据样本间的统计规律对样本集进行分析 , 常见任务如**聚类**等。



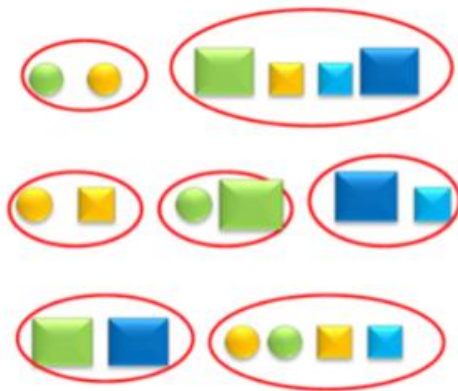
Clustering:

X: (颜色, 形状, 大小)

Data:

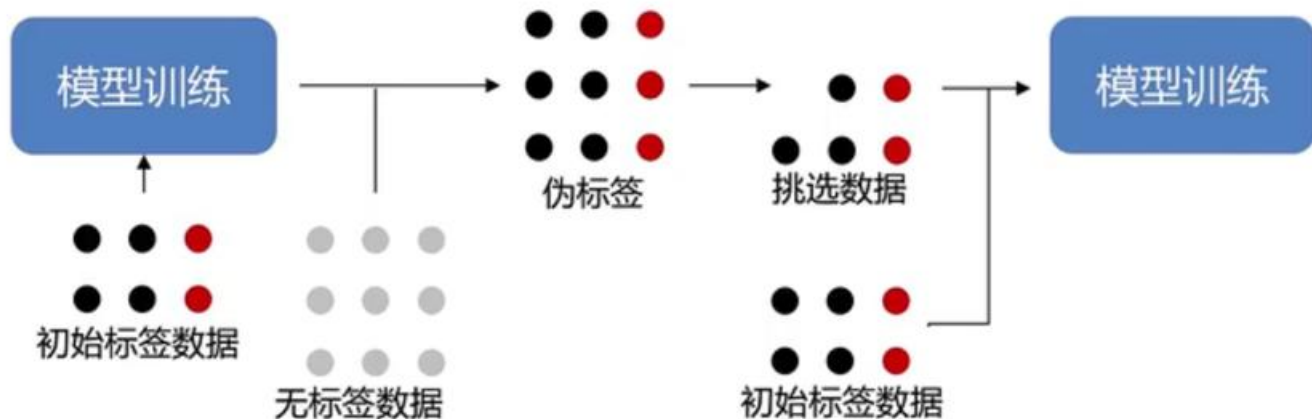


For all the data, Y=?



降维 (Dimensionality Reduction) : 如何将原高维空间中的数据点映射到低维度的空间中? 例如主成分分析 (PCA) 等

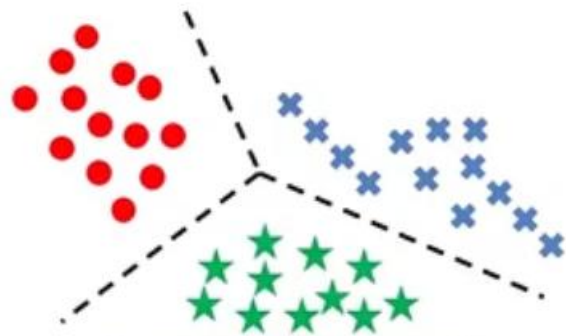
半监督学习 (Semi-supervised learning)：结合(少量的) 标注训练数据和 (大量的) 未标注数据来进行数据的分类学习。



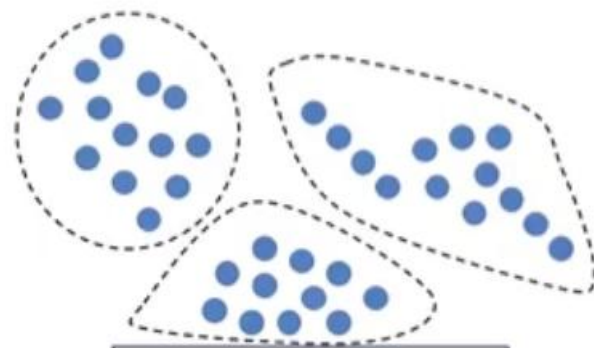
自训练 (Self-training)

半监督学习能够节约大规模数据标注带来的时间和费用开销

在标注数据量有限时，采用半监督学习有望训练得到更优的机器学习模型



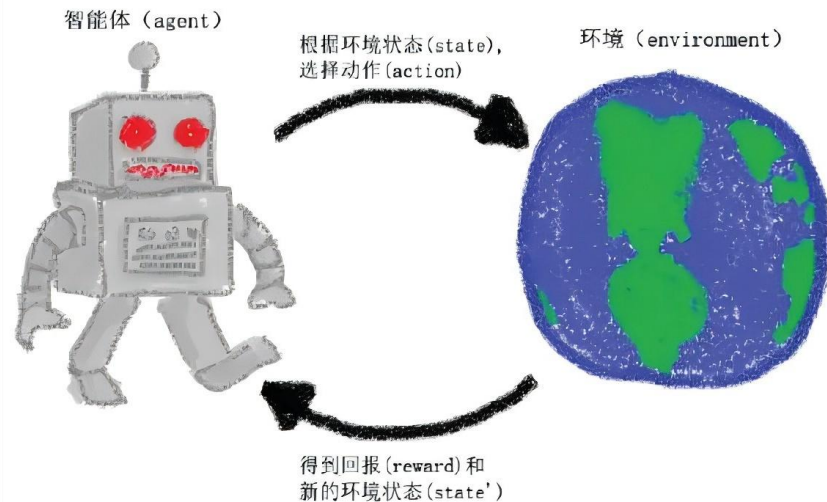
监督学习



无监督学习

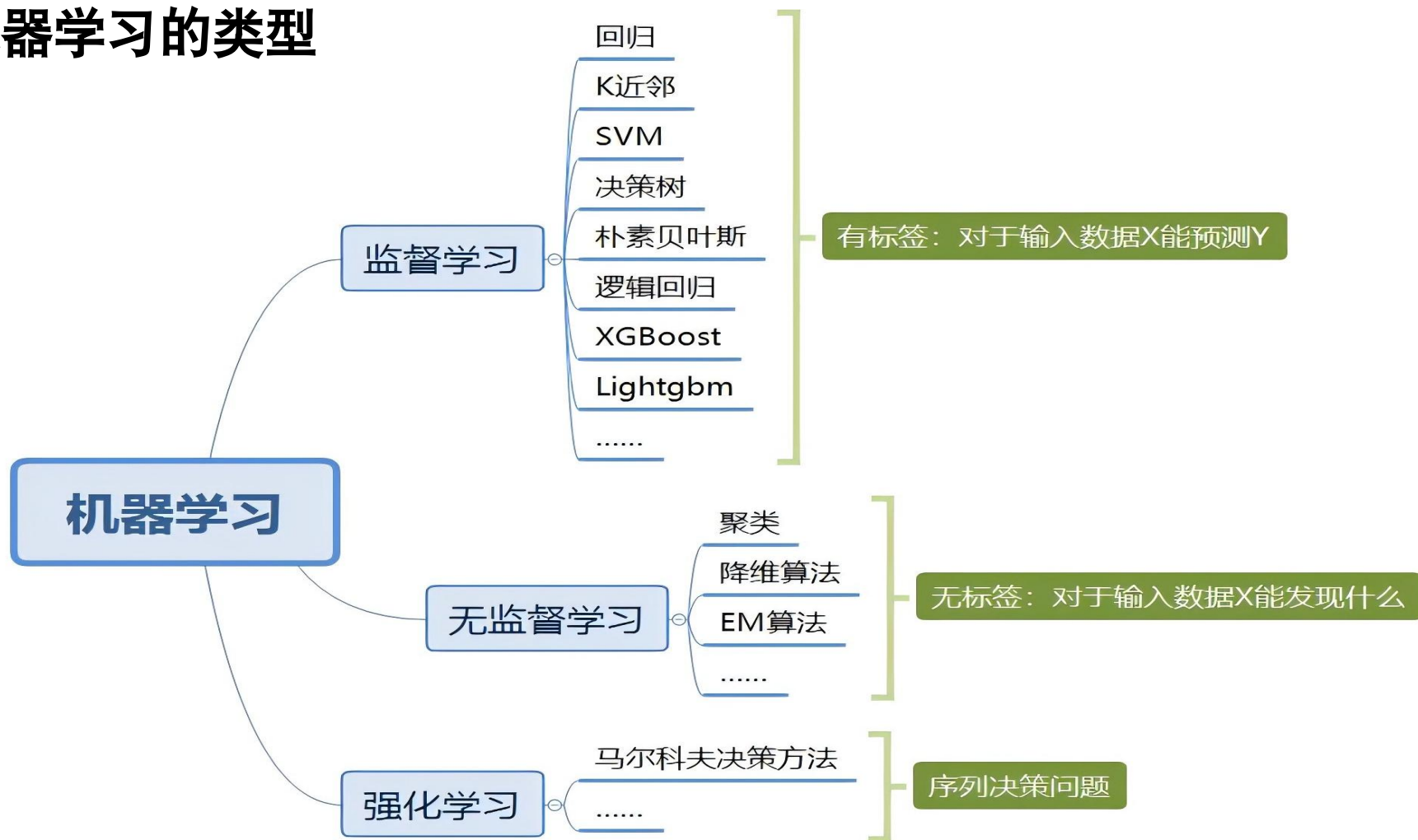


半监督学习

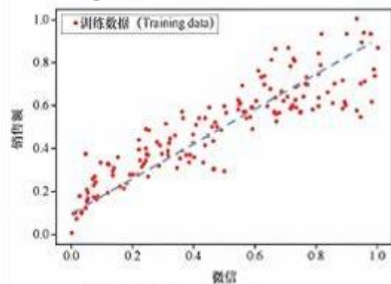


强化学习 (Reinforcement learning)

机器学习的类型



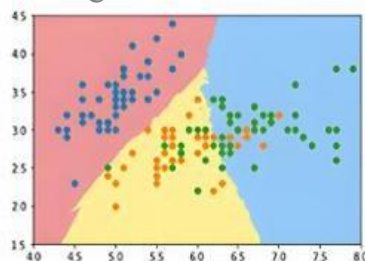
linear_model.LinearRegression



线性回归

Linear regression

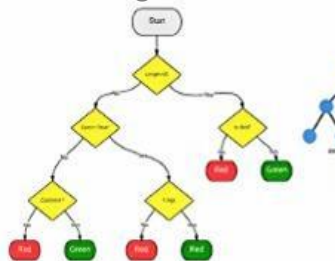
linear_model.LogisticRegression



逻辑回归

logistic regression

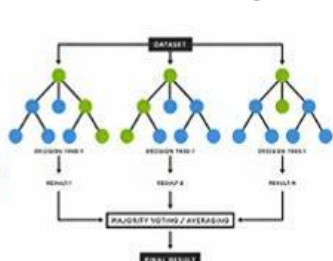
tree.DecisionTreeClassifier/~Regressor



决策树

decision tree

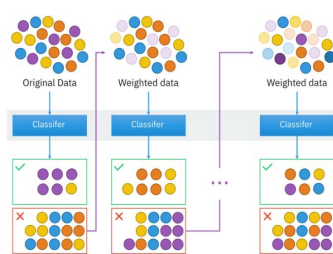
ensemble.RandomForestClassifier/~Regressor



随机森林

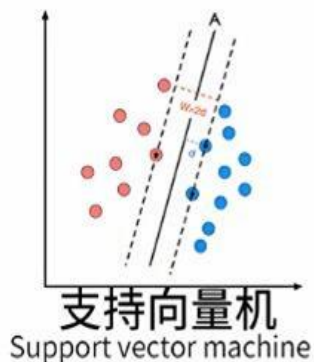
random forest

ensemble.AdaBoostClassifier/~Regressor



集成学习

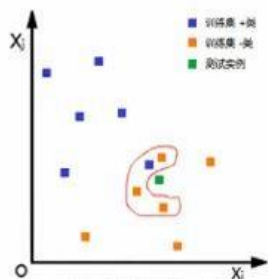
ensemble learning



支持向量机

Support vector machine

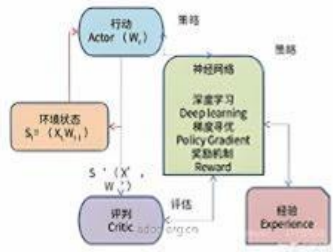
svm.SVC
svm.SVR



K-近邻算法

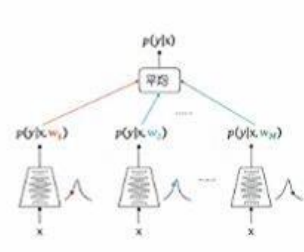
k-Nearest Neighbor

neighbors.KNeighborsClassifier/~Regressor



强化学习

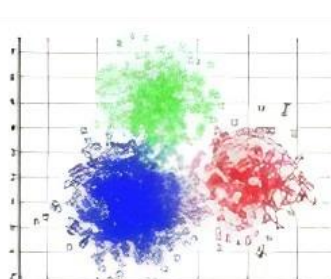
reinforcement learning



贝叶斯算法

Bayesian algorithm

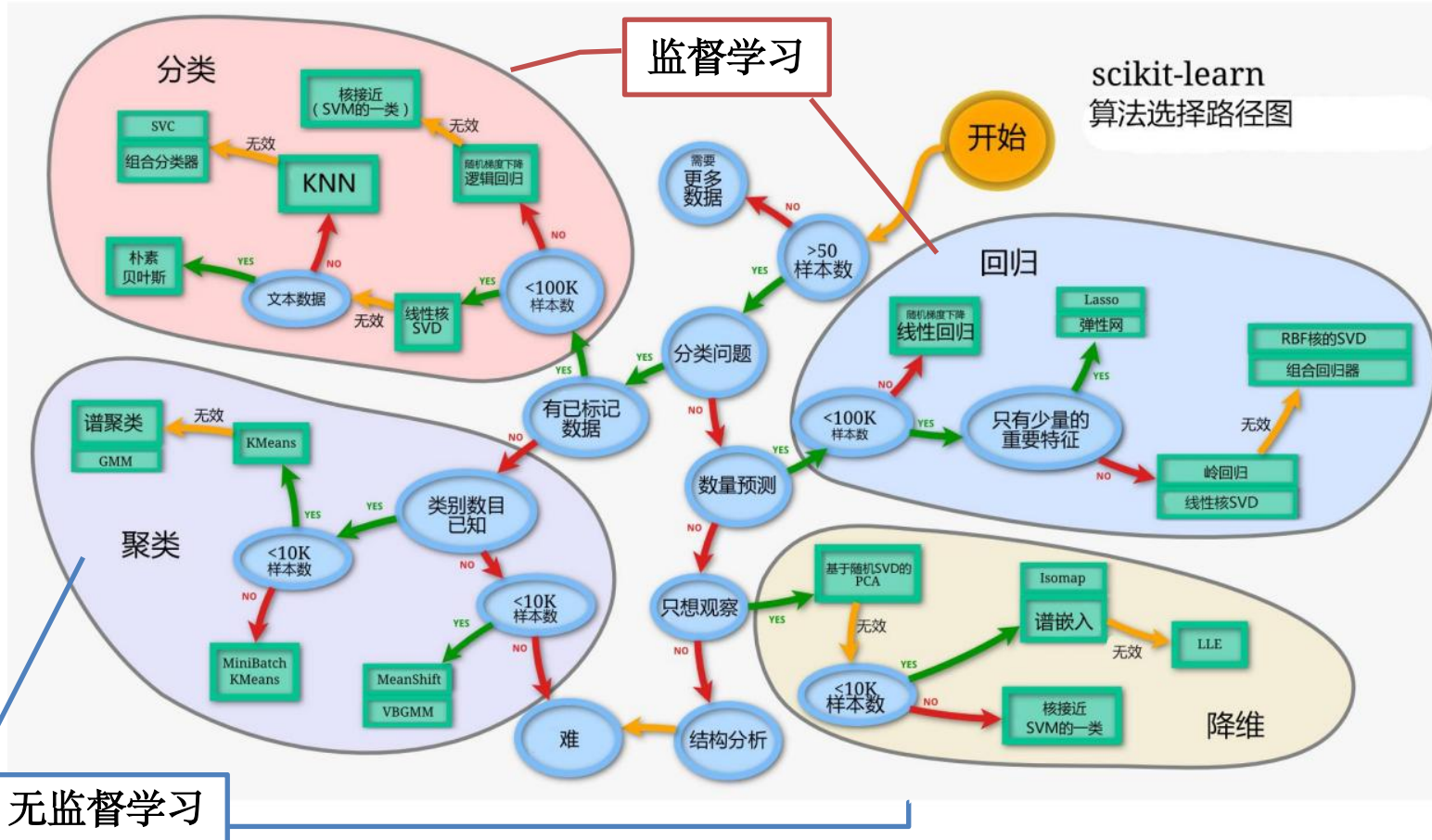
naive_bayes.GaussianNB/~MultinomialNB



k均值算法

k-means

cluster.KMeans



- **监督学习**算法：线性模型linear_model、支持向量机svm、树形模型tree等
- **聚类**算法cluster： K-Means 聚类、DBSCAN 聚类等
- **集成学习**算法 (ensemble)： 随机森林、梯度提升、AdaBoost 等
- **数据预处理** (preprocessing)： 数据标准化、归一化、编码分类变量等
- **模型选择与评估** (model_selection)： 数据集划分、交叉验证、超参数调优等
- **降维**工具 (decomposition)
- **特征提取**工具 (feature_extraction)
- **特征选择**工具 (feature_selection)
- **度量指标**模块 (metrics)



Scikit-Learn (sklearn) 是以 NumPy、SciPy 和 Matplotlib 为核心基础构建的

- Sklearn安装:

conda install scikit-learn

pip3 install scikit-learn

sklearn 中文文档 <https://scikit-learn.org.cn/>

sklearn 英文文档 <https://scikit-learn.org/>

对实际问题进行分析，确定问题的类型，
这会直接影响算法的选择、模型评估标准。

对训练好的模型进行测试，
评估模型是否达到需求。

对应问题选用合适的模型，
编写对应的模型代码。

问题定义

数据准备

模型选择
和开发

模型训练
和调优

模型测试
与评估

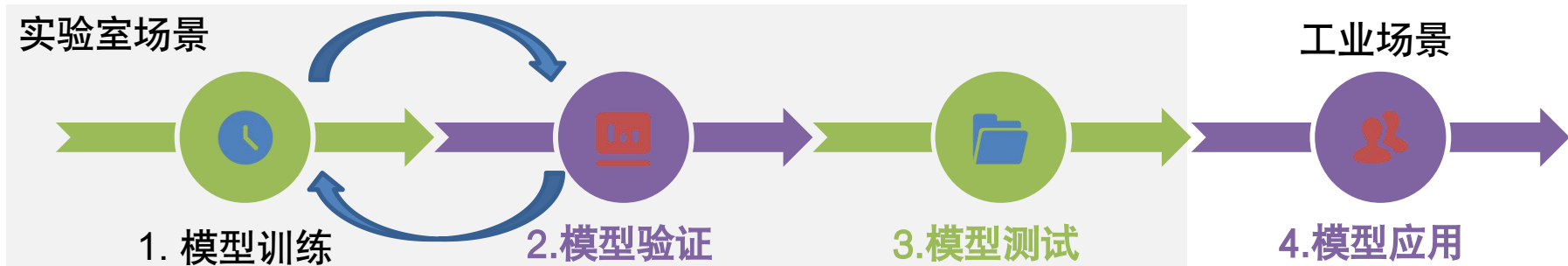
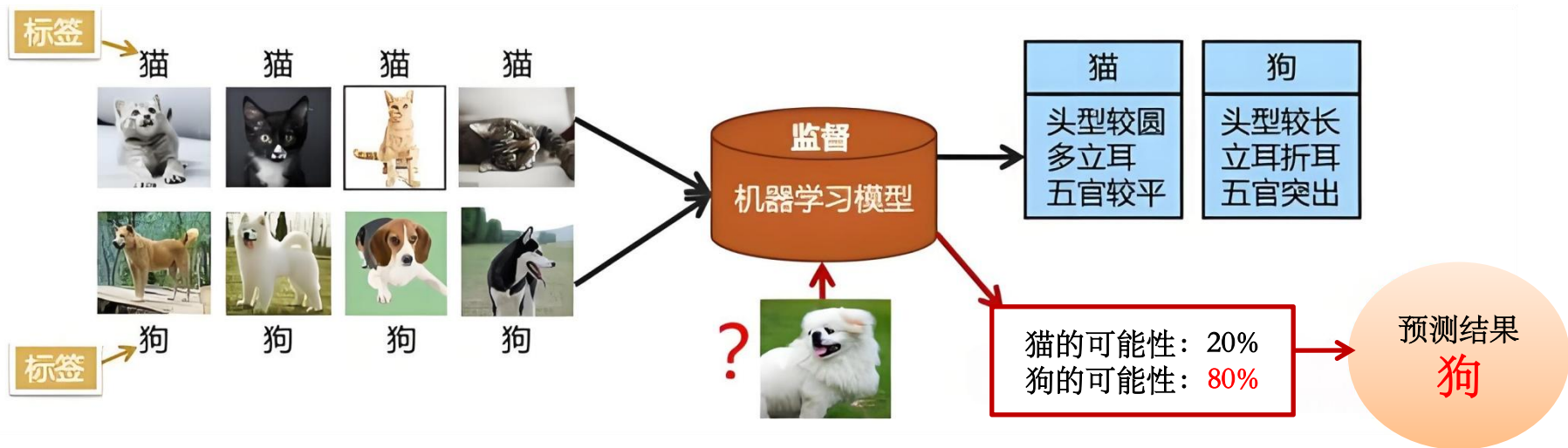
数据收集：下载、爬取。

数据预处理：数据清洗、特征工程、数据增强。

数据集定义和切分：训练、验证、测试。

使用数据集启动对模型的训练，
围绕模型目标进行调优。

监督学习示例：猫狗分类识别



2

机器学习的主要工作过程

数据搜集



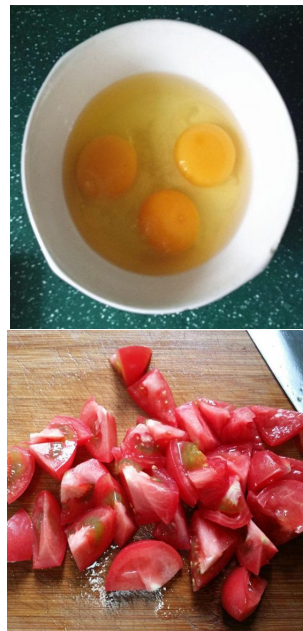
数据清洗



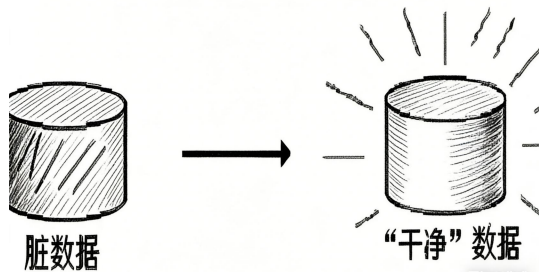
特征工程



数据建模



数据清洗



```
from sklearn.impute import SimpleImputer
```

- 缺失值处理
- 异常值处理
- 噪声处理

- 数据的完整性

例如人的属性中缺少性别、籍贯、年龄等；
解决方法：信息补全（使用身份证件号码推算性别、籍贯、出生日期、年龄等）；删除；

- 数据的合法性

例如获取数据与常识不符，年龄大于150岁；
解决方法：设置字段内容（日期字段格式为“2010-10-10”）；类型的合法规则（性别 in [男、女、未知]）

- 数据的一致性

例如不同来源的不同指标，实际内涵是一样的，或是同一指标内涵不一致；
解决方法：建立数据体系，包含但不限于指标体系、维度、单位、频度等

- 数据的唯一性

例如不同来源的数据出现重复的情况等；
解决方法：按主键去重（用sql或者excel“去除重复记录”） / 按规则去重（如不同渠道来的客户数据，可以通过相同的关键信息进行匹配，合并去重）

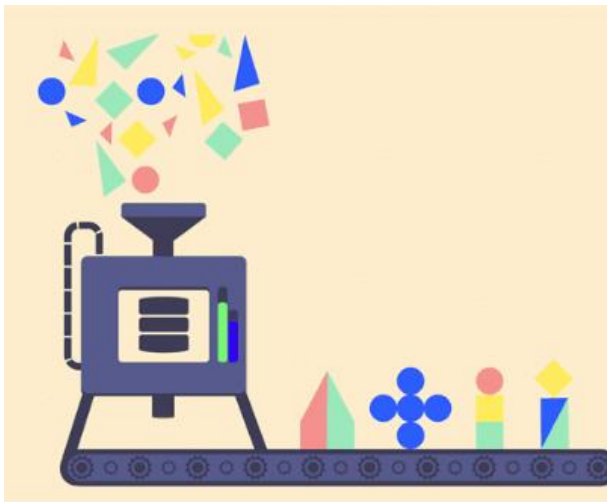
- 数据的权威性

例如出现多个来源的数据，且数值不一样；
解决方法：为不同渠道设置权威级别，如：在家里，首先得相信媳妇说的。...



特征工程：将原始数据转换为机器可理解的特征表示形式的过程。

目标：提取和选择对于机器学习算法来说最有信息量和预测能力的特征，从而改善模型的性能。



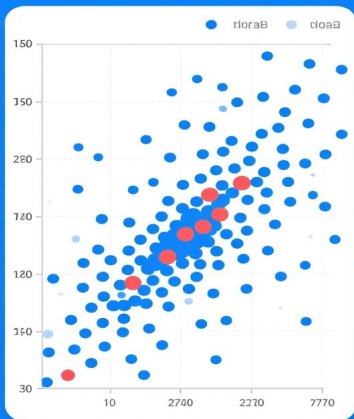
原始数据

数据特征

建模算法

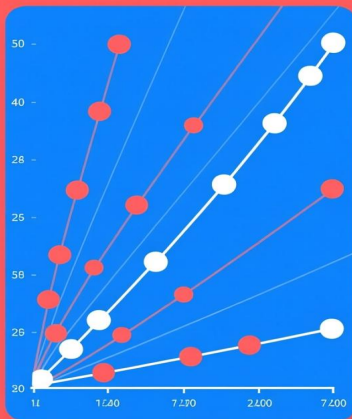
特征抽取

原始数据



原始数据

特征工程后

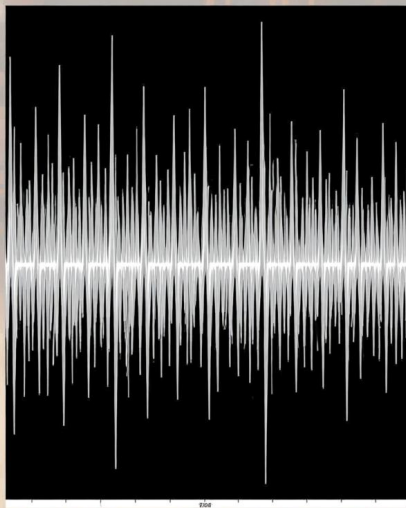


特征工程后

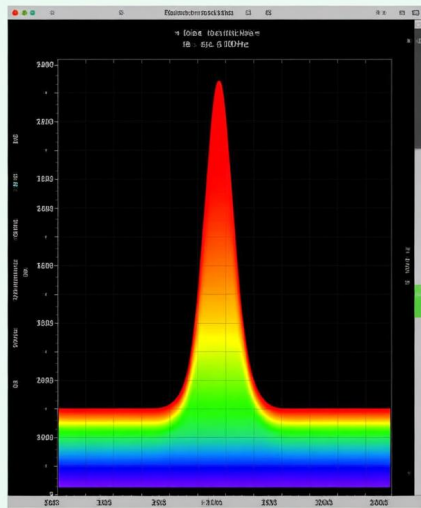
千问AI生成

Feature engineering

Raw sensor Data from machine

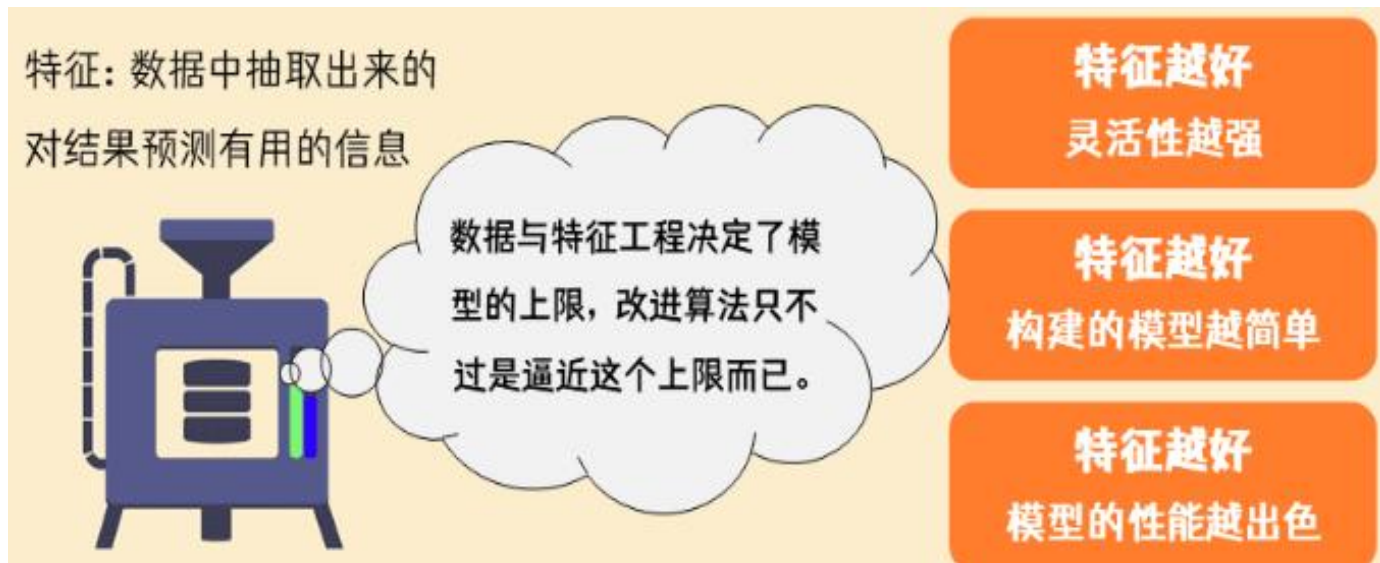
Raw Data
(Time Domain)

(FFT) Fourier Transform

Engineered Feature
(Frequency Domain)

千问AI生成

特征：数据中抽取出来的对结果预测有用的信息。



数据与特征工程决定了模型的上限，改进算法只不过是逼近这个上限而已。

```
from sklearn.feature_extraction import DictVectorizer
```

特征提取

将非数值数据（如文本、字典）转换为数值形式

特征构建

基于领域知识或数据组合生成新特征（如物理中的无量纲数）

特征选择

从原始特征中选择最重要的特征子集。

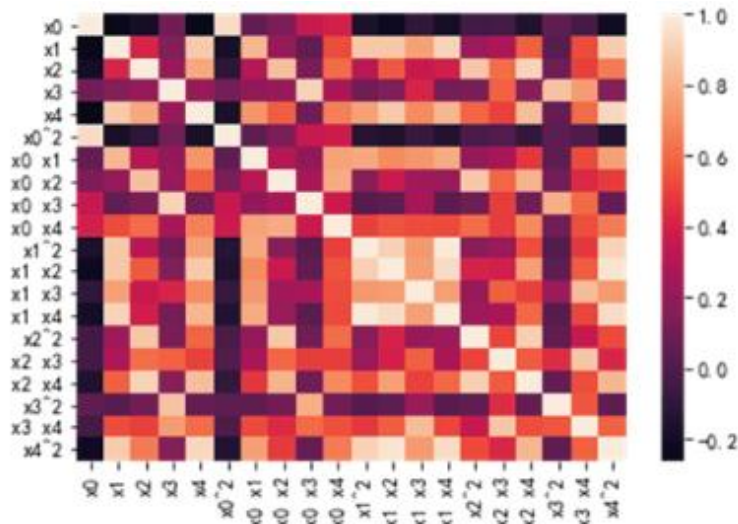
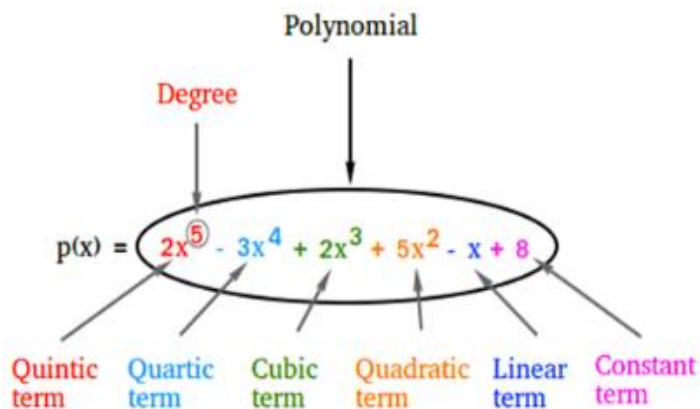
特征降维

减少特征维度，即减少相关性不大的特征数据或则方差较小的特征数据，从而减少模型的计算量。

多项式特征：

针对连续值特征，可以对几个特征通过四则运算构建多项式特征，以达到特征组合与高阶增强的作用。

```
from sklearn.preprocessing import PolynomialFeatures
```



★ 这些
属性特征
都有用吗?

特征选择



■ 例子 1：两类问题：男性、女性

数据特征：身高、体重、音频、头发长短、
出生日期、家庭住址、籍贯、专业

◆ 过滤法(Filter)

按照发散性或相关性对各特征进行评分，设定阈值完成特征选择。

■ 例子 2：高维数据的稀疏情况

数据 1: 1, 0, 0, 0, 0, 0, 1, 3, 0, 0, 0, 0, 0, 0
数据 2: 1, 0, 0, 0, 0, 0, 3, 3, 0, 0, 0, 0, 0, 0
数据 3: 1, 0, 1, 0, 0, 0, 5, 3, 0, 2, 0, 0, 0, 0

◆ 包裹法(Wrapper):

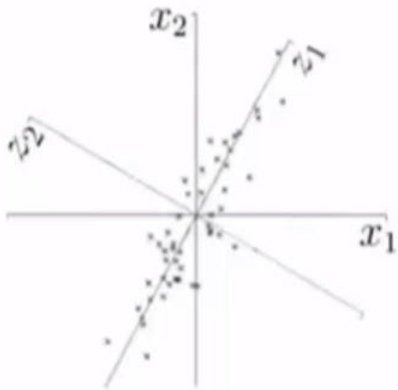
选定特定算法，然后通过不断的启发式方法来搜索特征。

```
from sklearn.feature_selection import RFE #递归特征消除
from sklearn.feature_selection import RFECV #带交叉验证的 RFE
```

特征选择完成后，可能由于特征矩阵过大，导致计算量大、训练时间长，因此**降低特征矩阵维度**也是必不可少的。

◆ 主成分分析(PCA)

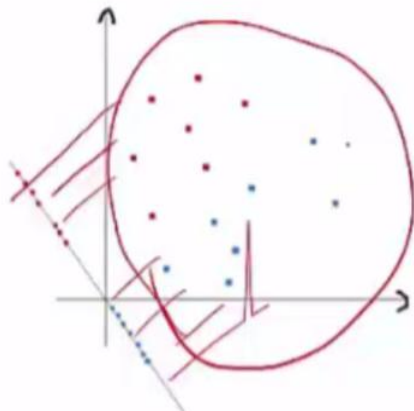
将原始特征空间映射到彼此正交的特征向量空间，在非满秩的情况下使用SVD分解来构建特征向量。



```
from sklearn.decomposition import PCA
```

◆ 线性判别分析(LDA)

给出一个标注了类别的数据集，投影到了一条直线之后，能够使得点尽量按类别区分开。



```
from sklearn.discriminant_analysis import  
LinearDiscriminantAnalysis as LDA
```

不同属性具有不同量级时会导致：①数量级的差异将导致量级较大的属性占据主导地位；②数量级的差异将导致迭代收敛速度减慢；③依赖于样本距离的算法对于数据的数量级非常敏感。

```
from sklearn.preprocessing  
import MinMaxScaler
```

◆ 标准化

通过减去均值然后除以方差（或标准差），将数据按比例缩放，使之落入一个小的特定区间。

$$x = (x - \mu) / \sigma$$

适用于：如果数据的分布本身就服从正态分布，就可以用这个方法。

```
from sklearn.preprocessing import StandardScaler
```

◆ 区间缩放

将属性缩放到一个指定的最大和最小值（通常是1-0）之间。

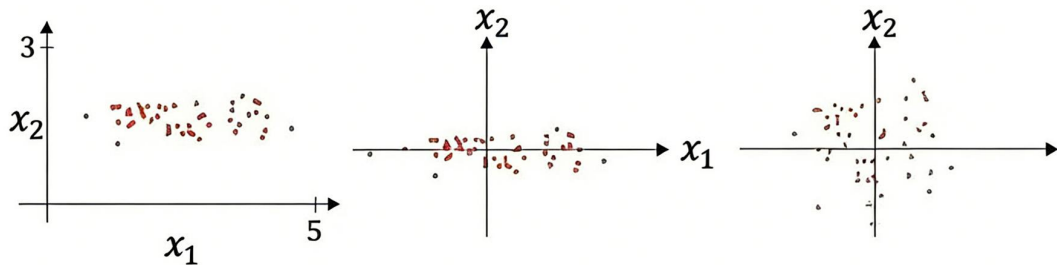
$$x = (x - \min) / (\max - \min)$$

◆ 归一化

将某一属性特征的模长转化成1。

$$x' = \frac{x}{\sqrt{\sum_j^m x[j]^2}}$$

```
from sklearn.preprocessing import Normalizer
```

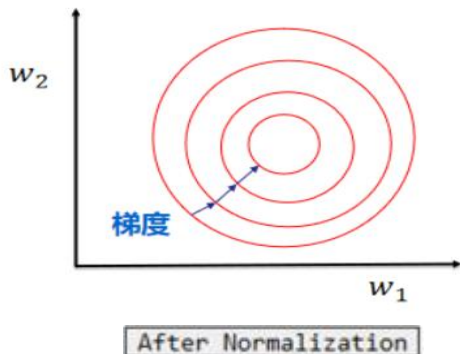
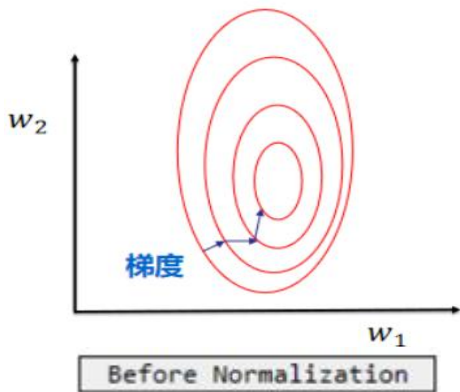


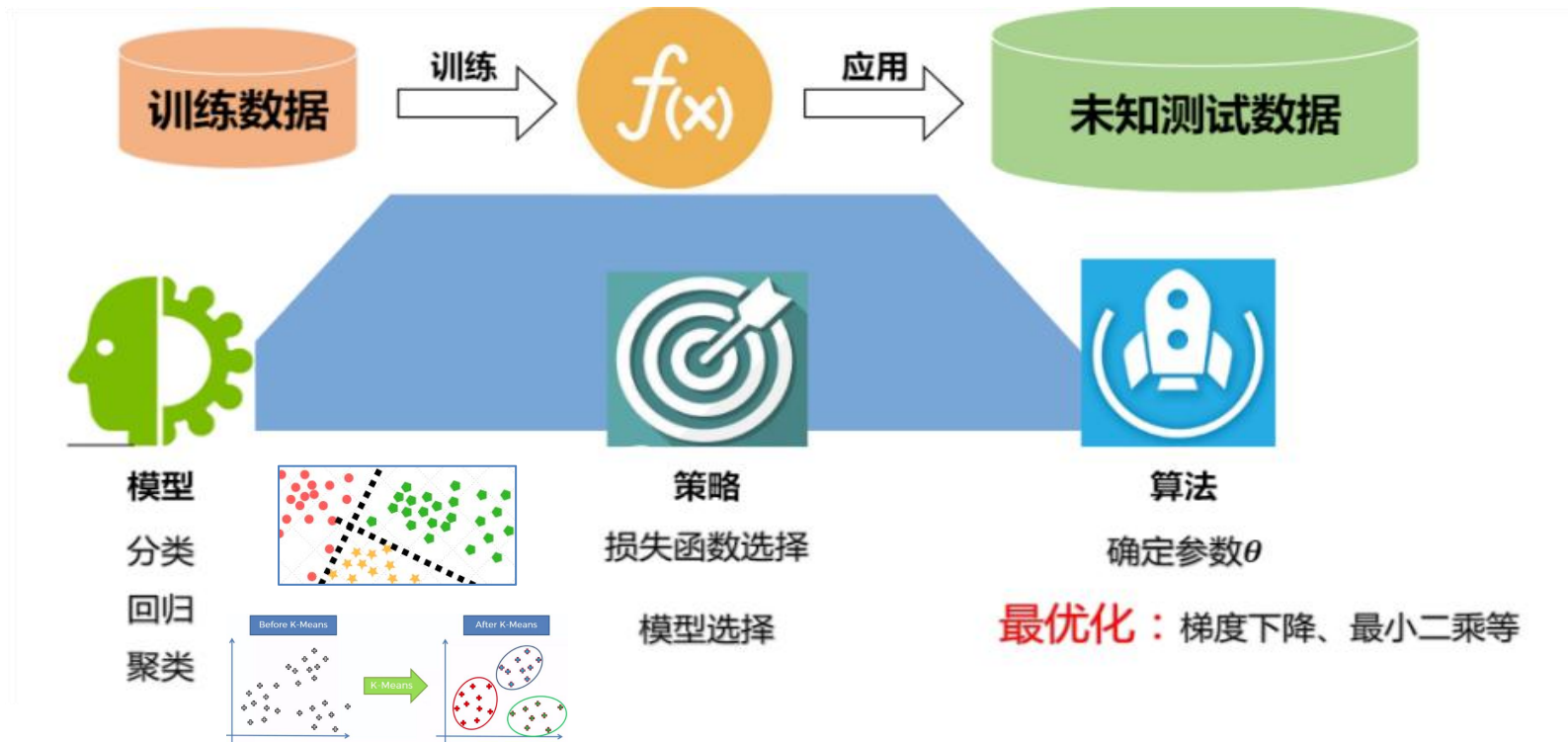
上图则是一个散点序列的标准化过程：原图 -> 减去均值 -> 除以标准差。

豆包AI生成

提升模型精度：不同维度之间的特征在数值上有一定比较性，可以大大提高分类器的准确性。

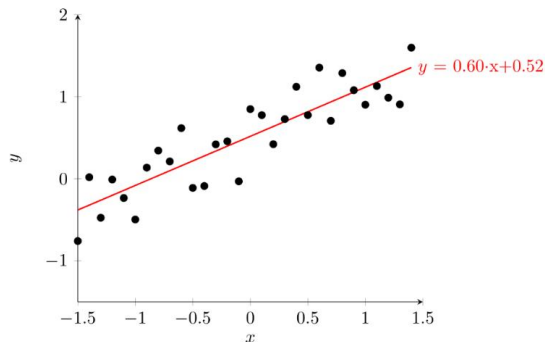
加速模型收敛：最优解的寻优过程明显会变得平缓，更容易正确的收敛到最优解。





- 学习模型

- 监督学习模型：回归模型、分类模型
- 无监督学习模型：聚类模型
- 半监督学习模型
- 强化学习模型



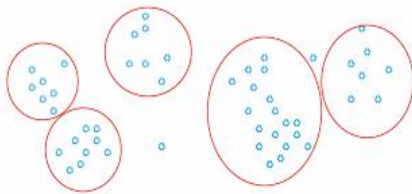
回归

- 学习准则

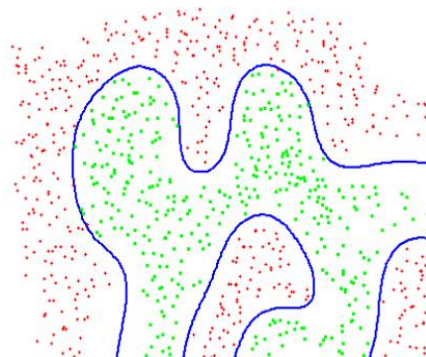
- 监督学习：最小化期望风险 (经验风险+结构风险)

- 优化算法

- 常用方法：梯度下降法



聚类



分类

- **损失函数 (Loss Function)**：度量单个样本预测的错误程度，损失函数值越小，模型就越好

- ✓ **平方损失**：对于单个样本，模型的预测值和真实值（标签）之间误差的平方，常用于回归问题

$$Loss(y_i, f(x_i)) = (y_i - f(x_i))^2$$

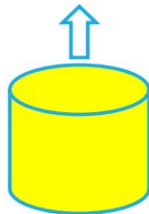
- ✓ **交叉熵损失**：衡量模型预测的概率分布与真实分布之间的偏差程度，常用于分类问题

$$Loss(y_i, P(y_i|x_i)) = -\log P(y_i|x_i)$$

- **代价函数 (Cost Function)**：在整个训练集上，所有样本误差的平均，也就是损失函数的平均，即**经验风险**最小化函数。
- **目标函数 (Object Function)**：最终需要优化的函数，等于经验风险 + 结构风险（也就是 Cost Function + 正则化项）。

训练映射函数 f

使得 $f(x_i)$ 预测结果尽量等于 y_i

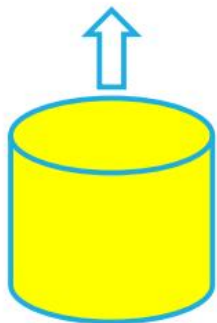


训练数据集

$(x_i, y_i), i = 1, \dots, n$

训练映射函数 f

使得 $f(x_i)$ 预测结果尽量等于 y_i



训练数据集

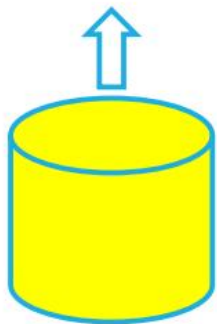
$(x_i, y_i), i = 1, \dots, n$

- 训练集中一共有 n 个标注数据，第 i 个标注数据记为 (x_i, y_i) ，其中第 i 个样本数据为 x_i ， y_i 是 x_i 的标注信息。
- 从训练数据中学习得到的映射函数记为 f ， f 对 x_i 的预测结果记为 $f(x_i)$ 。损失函数就是用来计算 x_i 真实值 y_i 与预测值 $f(x_i)$ 之间差值的函数。
- 很显然，在训练过程中希望映射函数在训练数据集上得到“损失”之和最小，即 $\min \sum_{i=1}^n \text{Loss}(f(x_i), y_i)$ 。

最小化经验风险

训练映射函数 f

使得 $f(x_i)$ 预测结果尽量等于 y_i



训练数据集

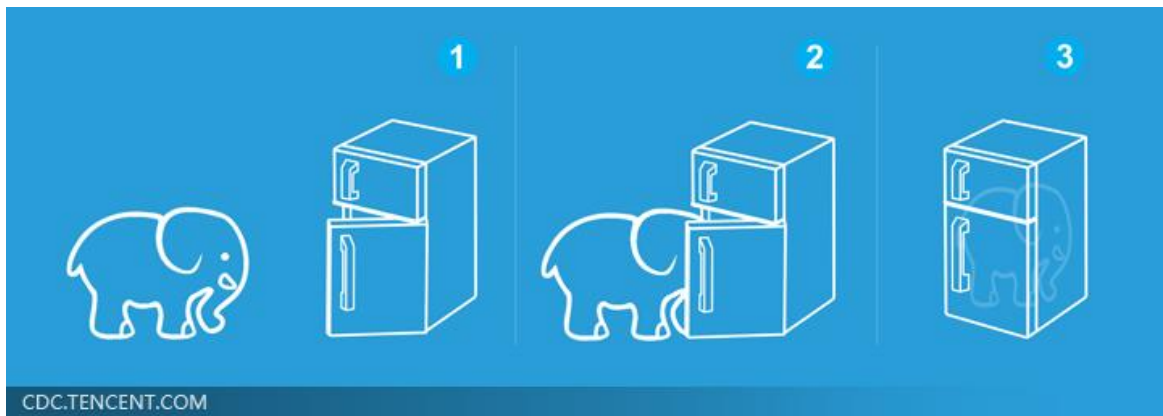
$(x_i, y_i), i = 1, \dots, n$

损失函数名称	损失函数定义
0-1损失函数	$Loss(y_i, f(x_i)) = \begin{cases} 1, f(x_i) \neq y_i \\ 0, f(x_i) = y_i \end{cases}$
平方损失函数	$Loss(y_i, f(x_i)) = (y_i - f(x_i))^2$
绝对损失函数	$Loss(y_i, f(x_i)) = y_i - f(x_i) $
对数损失函数/ 对数似然损失 函数	$Loss(y_i, P(y_i x_i)) = -\log P((y_i x_i))$

典型的损失函数

- 简单来讲，机器学习模型通过优化损失函数来调整内部参数，使得模型的预测更准确。

- 1) 定义损失函数
- 2) 计算误差
- 3) 调整参数
- 4) 停止优化



- **梯度下降法**：通过迭代优化的方式，让模型不断调整参数，逐步降低损失函数的值，从而找到最优解。
 - 1) 观察四周，找出**最陡峭、下坡速度最快**的方向；
 - 2) 沿着所选方向迈出一小步；
 - 3) 不断调整重复上述步骤，且每次都微调方向，向更低处移动，直到周围没有更低的位置。



梯度下降法 (Gradient Descent Method), 也叫作最速下降法 (Steepest Descend Method), 经常用来求解无约束优化的最小值问题.

对于函数 $f(\mathbf{x})$, 如果 $f(\mathbf{x})$ 在点 $\mathbf{x}_t$ 附近是连续可微的, 那么 $f(\mathbf{x})$ 下降最快的方向是 $f(\mathbf{x})$ 在 $\mathbf{x}_t$ 点的梯度方向的反方向.

根据泰勒一阶展开公式, 有

$$f(\mathbf{x}_{t+1}) = f(\mathbf{x}_t + \Delta \mathbf{x}) \approx f(\mathbf{x}_t) + \Delta \mathbf{x}^\top \nabla f(\mathbf{x}_t). \quad (\text{C.7})$$

要使得 $f(\mathbf{x}_{t+1}) < f(\mathbf{x}_t)$, 就得使 $\Delta \mathbf{x}^\top \nabla f(\mathbf{x}_t) < 0$. 我们取 $\Delta \mathbf{x} = -\alpha \nabla f(\mathbf{x}_t)$. 如果 $\alpha > 0$ 为一个够小数值时, 那么 $f(\mathbf{x}_{t+1}) < f(\mathbf{x}_t)$ 成立.

这样我们就可以从一个初始值 $\mathbf{x}_0$ 出发, 通过迭代公式

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha_t \nabla f(\mathbf{x}_t), \quad t \geq 0. \quad (\text{C.8})$$

生成序列 $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots$ 使得

$$f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq f(\mathbf{x}_2) \geq \dots \quad (\text{C.9})$$

《神经网络与深度学习》

Neural Networks and Deep Learning

<https://nndl.github.io/>

邱锡鹏

xpqiu@fudan.edu.cn

2021 年 5 月 18 日

C.2.2 梯度下降法

梯度下降法：

- 1) 随机初始化模型参数 θ
- 2) 计算目标函数的参数梯度
- 3) 更新模型参数
- 4) 判断收敛

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial \mathcal{R}(\theta)}{\partial \theta_t}$$

$$= \theta_t - \alpha \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})}{\partial \theta}$$

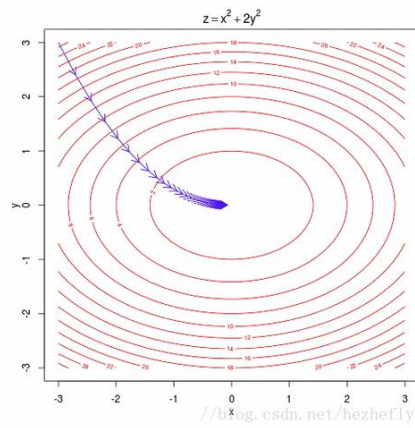
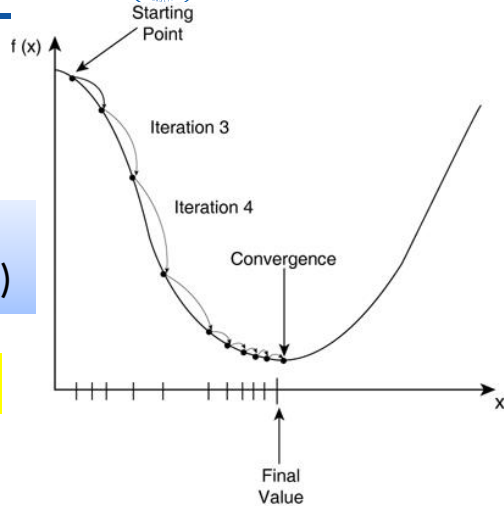
目标函数 R
(这里是代价函数)

损失函数 L

步长 α 中也叫作学习率

$x^{(i)}$: 第 i 个样本数据

$y^{(i)}$: 第 i 个样本的标注信息 (真实值)



以下哪项不是梯度下降法的步骤？

- ☐ A 随机初始化模型参数
- ☐ B 计算梯度
- ☐ C 调整学习率
- ☐ D 更新模型参数

提交

以下哪项不是梯度下降法的步骤？

- ☐ A 随机初始化模型参数
- ☐ B 计算梯度
- ☒ C 调整学习率
- ☐ D 更新模型参数

提交

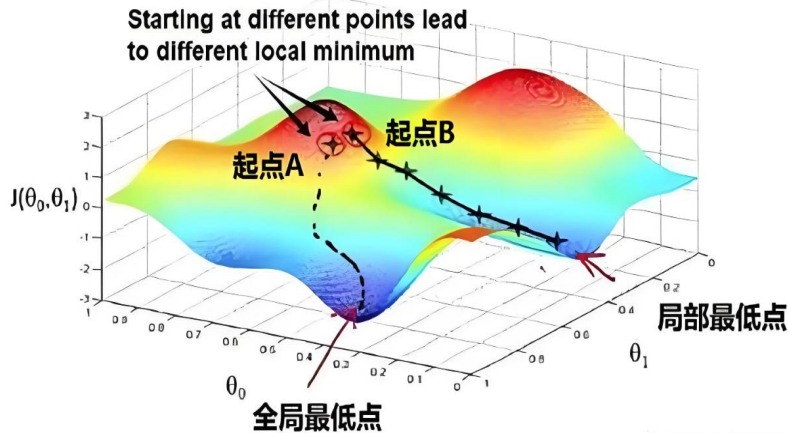
- 梯度下降法的局限：

- 局部最优解

- 学习率的选择：

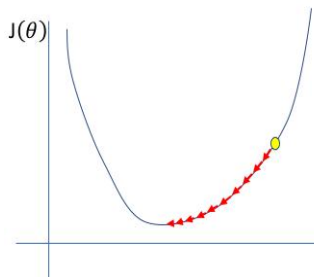
学习率太大，可能会跳过最低点；

学习率太小，优化过程会非常缓慢



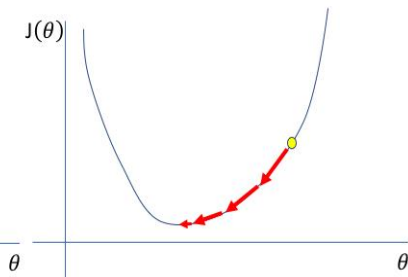
•使用自适应学习率的优化器：
如 Adam、RMSprop 等。它们能根据梯度历史信息调整每个参数的学习率，有时能帮助“跳出”一些较浅的局部最小值。

Too low



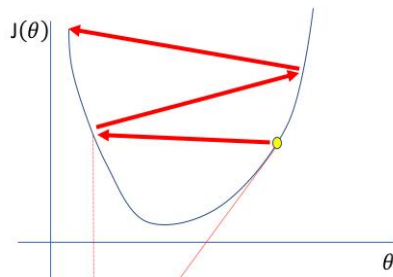
A small learning rate requires many updates before reaching the minimum point

Just right



The optimal learning rate swiftly reaches the minimum point

Too high

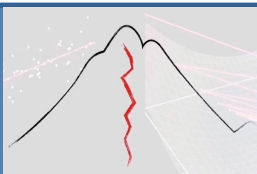


Too large of a learning rate causes drastic updates which lead to divergent behaviors

➤ 梯度下降法的类型:



批量梯度下降 (BGD)



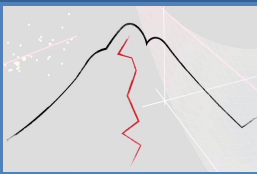
每步使用**所有训练样本**来计算梯度

随机梯度下降 (SGD)

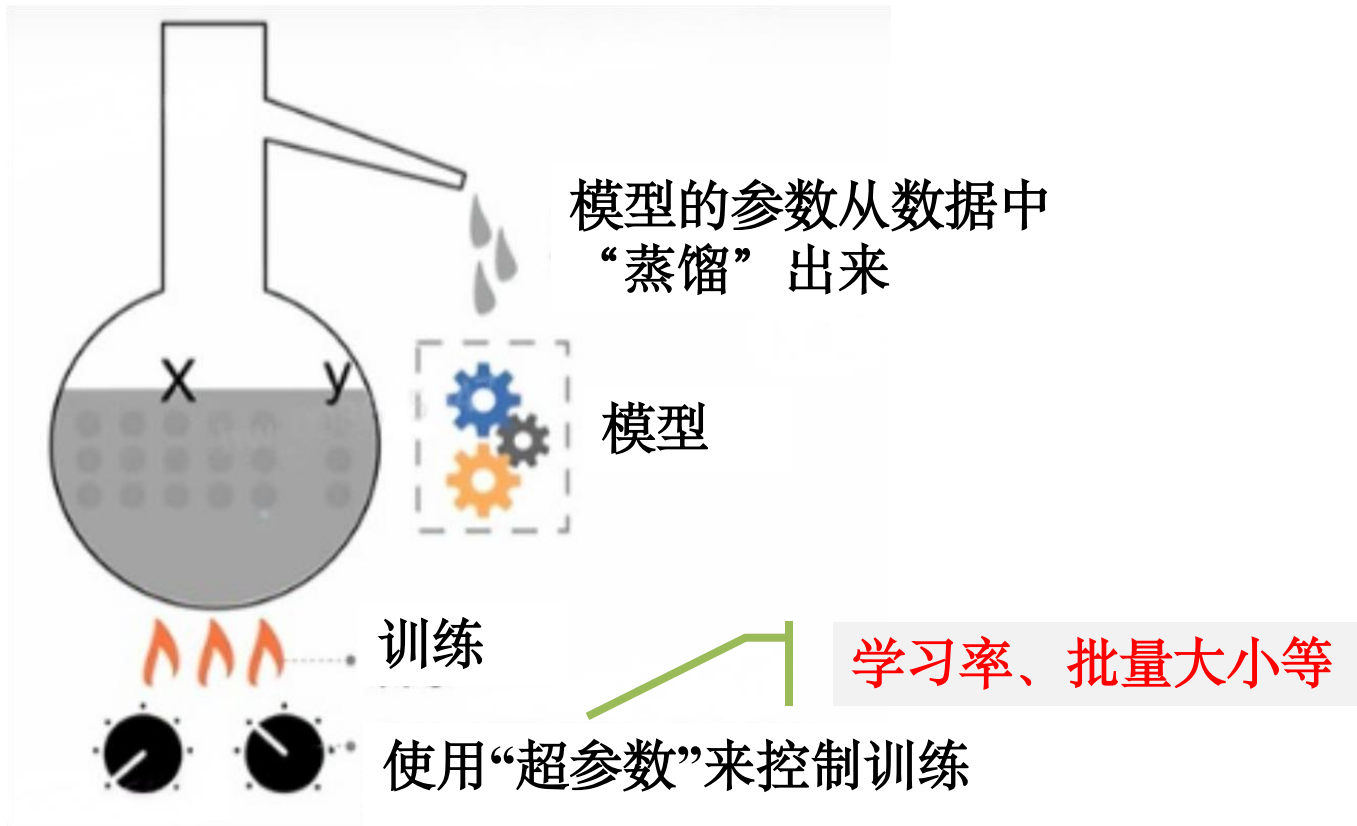


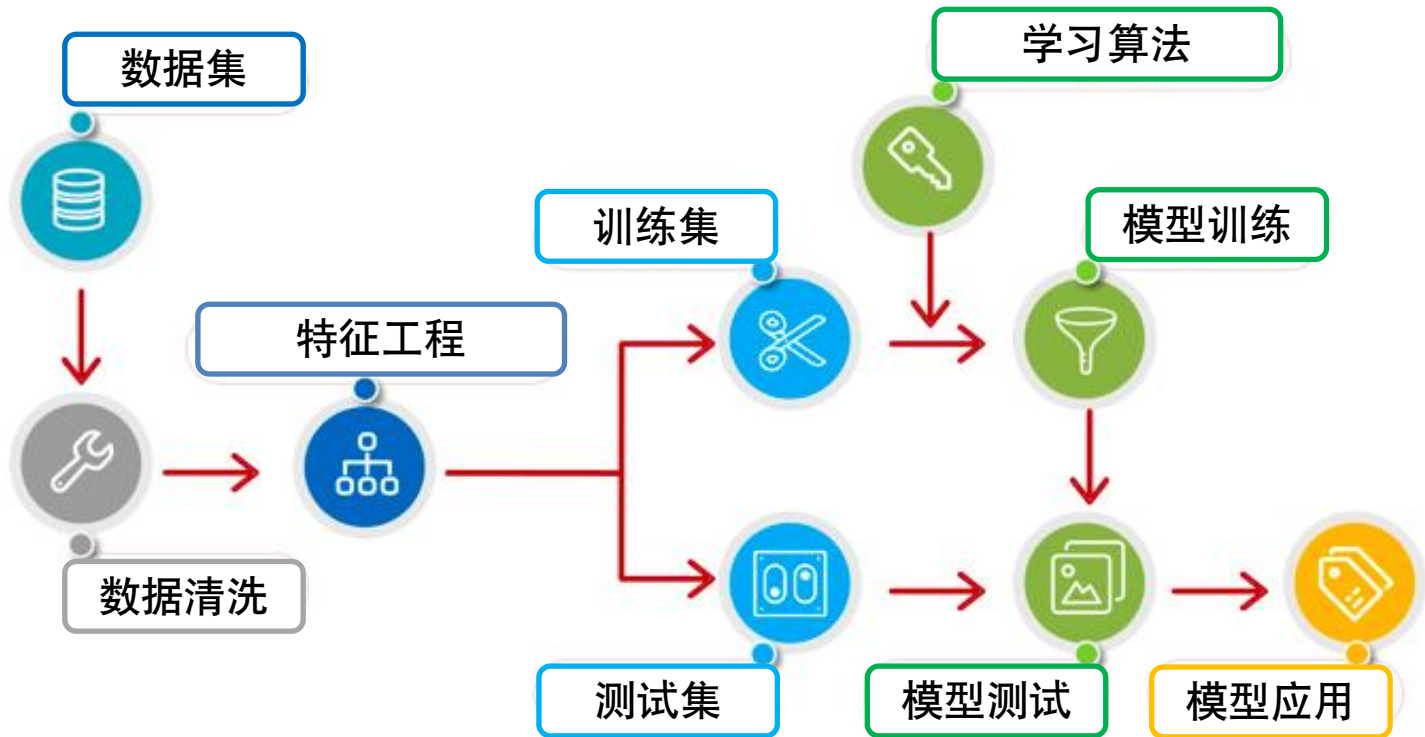
每步只使用**一个样本**来计算梯度

小批量梯度下降 (MBGD)

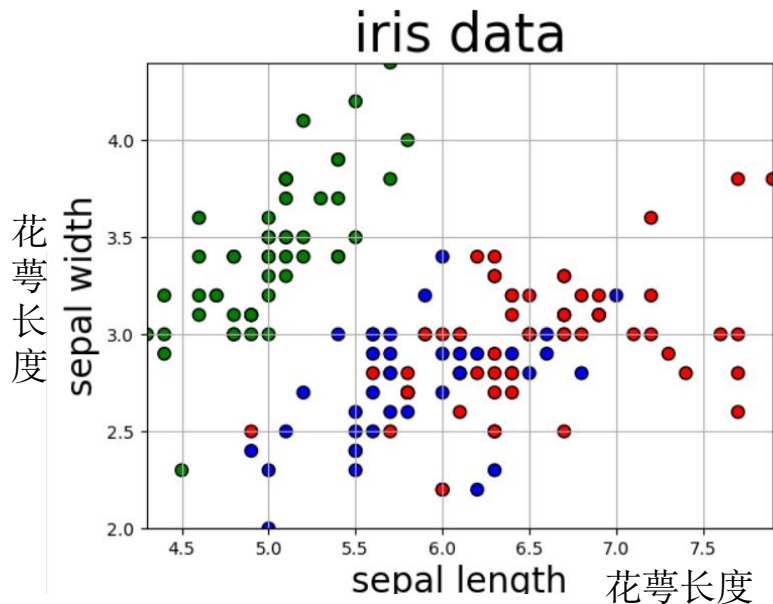


每步只使用**一小批样本**来计算梯度

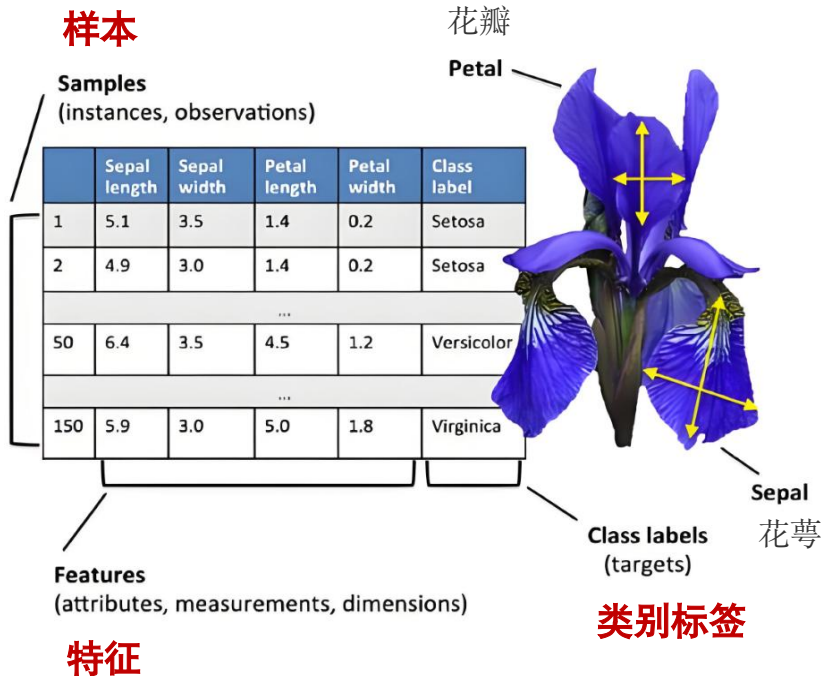




鸢尾花Iris数据集（由Fisher, 1936收集整理）：包含3种类别，每类各50个记录，每条记录有花萼长度、花萼宽度、花瓣长度、花瓣宽度



g: 山鸢尾(setosa), b: 变色鸢尾(versicolor),
r: 维吉尼亚鸢尾(virginica)



from sklearn.preprocessing import LabelEncoder, OneHotEncoder
标签编码, 独热编码

- **训练集 (train dataset)** : 用来构建机器学习模型
- **验证集 (validation dataset)** : 辅助构建模型, 用于在构建过程中评估模型, 提供无偏估计, 进而调整模型参数
- **测试集 (test dataset)** : 用于评估

数据划分: **训练集、验证集、测试集**

机器学习: 60%、20%、20%

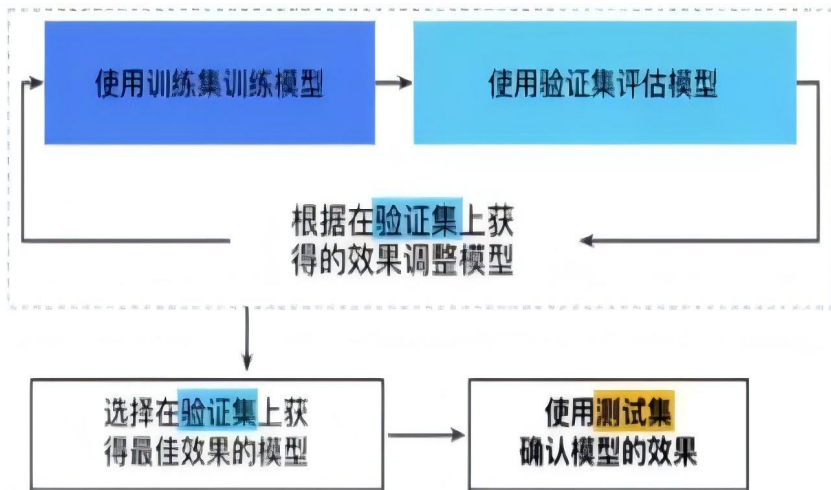
70%、10%、20%

深度学习: 98%、1%、1% (假设百万条数据)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
#拆分数据集
print("加载数据中...")
dataset = load_iris()
```

```
#dataset.data: 特征; dataset.target: 标签
X_train, X_test, y_train, y_test =
train_test_split( dataset.data[:, :2], dataset.target,
random_state=3, test_size=0.3)
```

test_size: 测试集样本数占总样本的比例



■ 模型训练

- 利用已知样本，建立起特征空间到目标空间的映射关系、或者是利用已知样本拟合 (fit) 特征与目标的关系。
- 在sklearn中，这个过程用fit方法来表示
model.fit(feature_in_train, target_in_train)

```
from sklearn import svm #导入支持向量机SVM模型
```

```
#创建一个 SVM 对象，设置好参数
```

```
classifier_svm = svm.SVC(C=0.1, kernel='linear', decision_function_shape='ovr')
```

```
classifier_svm.fit(X_train, y_train) #调用fit函数，模型训练
```

- 模型测试与模型评估

- 检验训练后的模型能否成功预测某一输入下的输出
- 在sklearn中，这个过程用predict方法来表示

pred = **model.predict**(feature_in_test)

```
y_pred = classifier_svm.predict(X_test) #预测
```

```
from sklearn import metrics
```

```
# 计算混淆矩阵
```

```
conf_matrix = metrics.confusion_matrix(y_test, y_pred)
```

```
#求正确率、F1分数、精确率、召回率
```

```
acc = metrics.accuracy_score(y_test, y_pred)
```

```
f1_macro = metrics.f1_score(y_test, y_pred, average='macro')
```

```
precision_macro = metrics.precision_score(y_test, y_pred, average='macro')
```

```
recall_macro = metrics.recall_score(y_test, y_pred, average='macro')
```

- 模型评估: 性能指标——分类问题

二分类混淆矩阵

真实结果	预测结果	
	正例	反例
正例	TP	FN
反例	FP	TN

TP (True Positive): 预测为正, 实际也为正。(正确检测)

FN (False Negative): 预测为负, 但实际为正。(漏报)

FP (False Positive): 预测为正, 但实际为负。(误报)

TN (True Negative): 预测为负, 实际也为负。(正确拒绝)

- 正确率: 分类正确的样本数占样本总数的比例

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- 模型评估: 性能指标——分类问题

二分类混淆矩阵

真实结果	预测结果	
	正例	反例
正例	TP	FN
反例	FP	TN

- 正确率: 分类正确的样本数占样本总数的比例

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- 查准率 (也称**精确率**): 预测结果为正的样本有多少实际也为正

$$\text{查准率 } P = \frac{TP}{TP + FP}$$

- 查全率 (也称**召回率**): 实际为正的样本中有多少样本的分类结果也为正

$$\text{查全率 } R = \frac{TP}{TP + FN}$$

- F1分数: 综合考虑查准率与查全率的度量

$$F_1 = \frac{2 \times P \times R}{P + R}$$

```
print("混淆矩阵: \n", conf_matrix)
print(f'测试结果: F1: {f1_macro:.2f}, P: {precision_macro:.2f}, R: {recall_macro:.2f},
acc: {acc:.2f}')
```

绘制混淆矩阵热力图

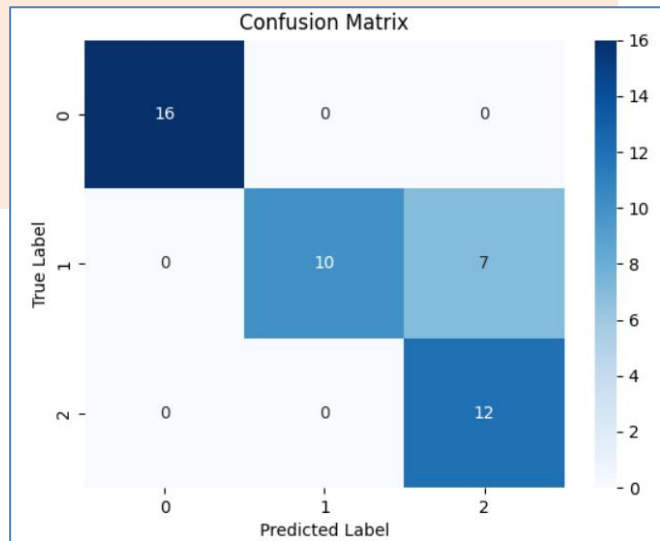
```
import seaborn as sns
import matplotlib.pyplot as plt
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

先为每个类别独立计算其精确率 (Precision)，然后取这些精确率的算术平均值

混淆矩阵:

```
[[16  0  0]
 [ 0 10  7]
 [ 0  0 12]]
```

测试结果: F1: 0.84, P: 0.88, R: 0.86, acc: 0.84



假设你想建立一个关于陨石撞地球的分类模型，现在有100个训练样本，其中**99个负样本**（**陨石没撞地球**），**1个正样本**（**陨石撞了地球**）。

经过训练，得到了 99% 的正确率，你是应该感到高兴吗？

- ☐ A 是
- ☐ B 否
- ☐ C 不知道



提交

□ 数据不平衡 (imbalance)

- ✓ **指数据集的类别分布不均。**比如说一个二分类问题，100个训练样本，比较理想的情况是正类、负类样本的数量相差不多；而如果正类样本有99个、负类样本仅1个，就意味着存在类不平衡。
- ✓ 此时预测时就算全部为正，准确率也可以达到99%，这并**不能反映模型的好坏。**

面临**不平衡数据集**的时候，传统的机器学习模型的评价方法不能精确地衡量模型的性能

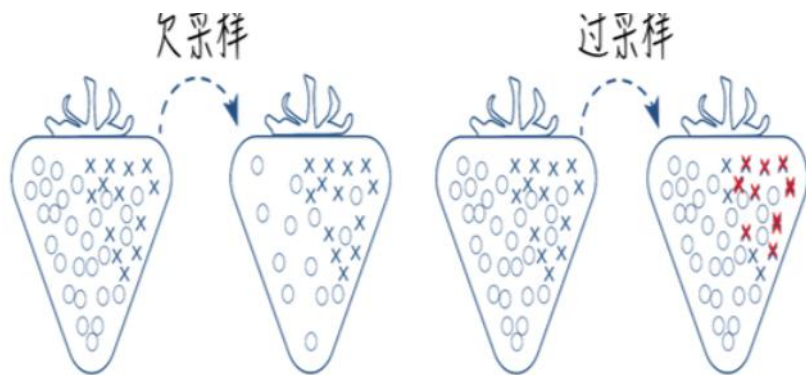
□ 解决方法

✓ 过采样 (Over-Sampling)

通过随机复制少数类来增加其中的实例数量，从而可增加样本中少数类的代表性。

✓ 欠采样 (Under-Sampling)

通过随机地消除占多数的类的样本来平衡类分布；直到多数类和少数类的实例实现平衡。



二分类混淆矩阵

真实结果	预测结果	
	正例	反例
正例	TP	FN
反例	FP	TN

真正率TPR/灵敏度(Sensitivity)/召回率(Recall):

$$\text{sensitivity} = TPR = \frac{TP}{TP + FN} \quad \text{识别正样本的能力}$$

特异性/真负率TNR (True Negative Rate) :

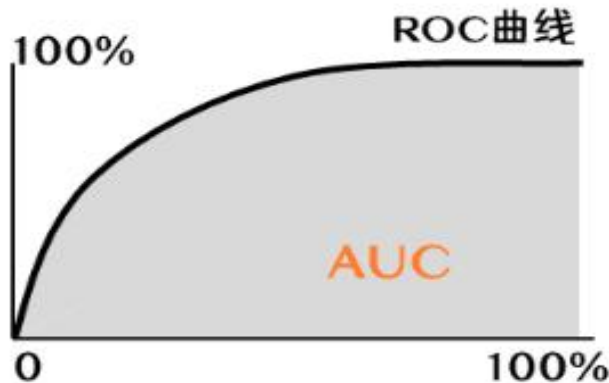
$$\text{specificity} = \frac{TN}{TN + FP} \quad \text{识别负样本的能力}$$

真实需求: 即能找正样本, 也能找出负样本

TPR (True Positive Rate)

$$TPR = \frac{TP}{TP + FN}$$

AUC (Area Under the ROC Curve)



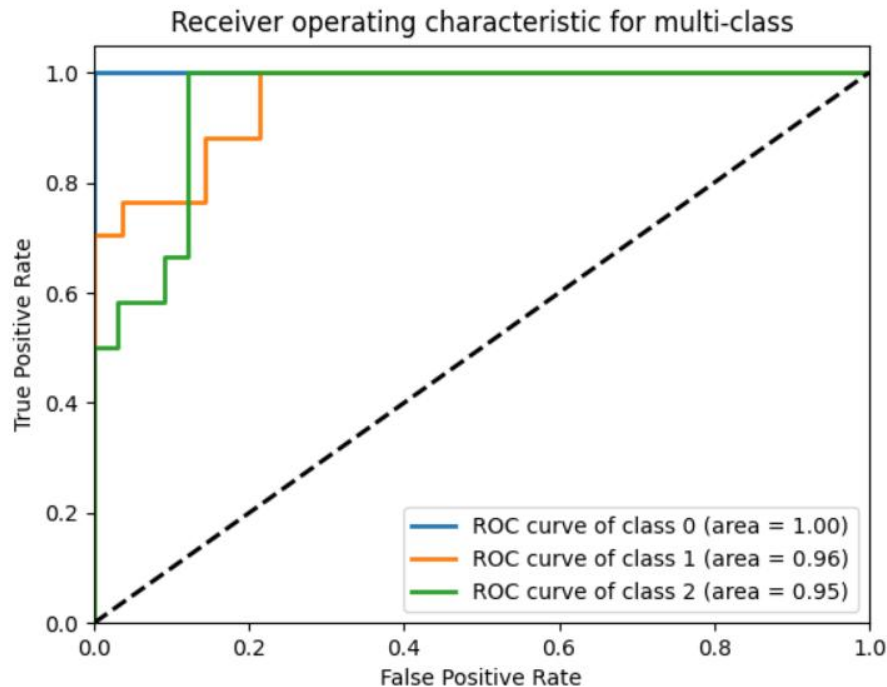
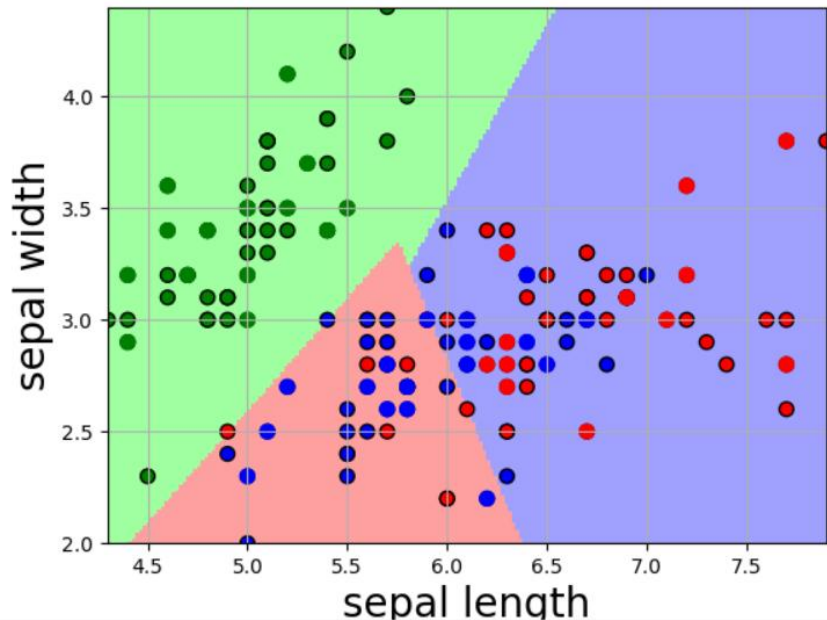
FPR (False Positive Rate)

$$FPR = \frac{FP}{FP + TN}$$

假正率FPR/误检率衡量模型误报的程度。FPR越低, 特异性越高, 说明误报越少。

- 模型评估: 性能指标——分类问题

svm in iris data classification



- 模型评估: 性能指标——回归问题

名称	公式
均方误差: MSE	$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$
均方根误差: RMSE	$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}$
平均绝对误差: MAE	$MAE = \frac{1}{m} \sum_{i=1}^m y_i - \hat{y}_i $
R^2 : R-squared	$R^2 = 1 - \sum_{i=1}^m \frac{(y_i - \hat{y}_i)^2}{(y_i - \bar{y})^2}$

y_i : 测试集上的真实值

$\hat{y}_i$: 模型的预测值

$\bar{y}$: 数据集的均值

m : 样本数

R^2 越接近于1, 表示当前的回归模型对数据的解释越好, 越能精确描述数据的真实分布

- 模型评估: 评估方法

常见的评估方法: 留出法 (Hold-out) 、交叉验证法 (Cross Validation) 等

留出法

从训练数据中保留出验证样本集, 这部分数据不用于训练, 而用于模型评估。

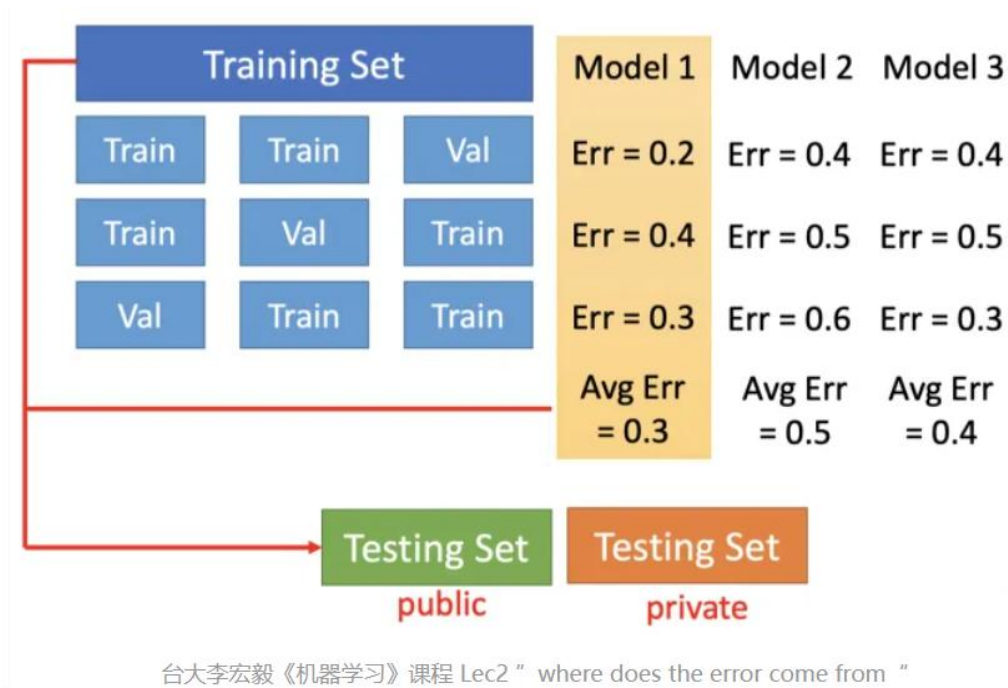
交叉验证法

k 折交叉验证对 k 个不同分组训练的结果进行平均来减少方差, 模型评估结果更加稳定。

k 折交叉验证:

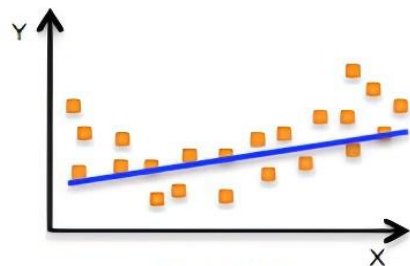
数据量小的时候, k 设大一点; 数据量大的时候, k 可以设小一点;

k = 3



■ 模型调优

希望找到对当前问题表达能力好，且模型复杂度较低的模型，能够很好地适用于新数据，即泛化能力强



Underfitting

欠拟合

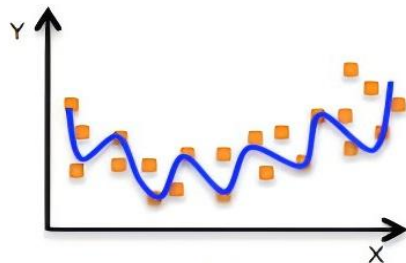
$$\theta_0 + \theta_1 x$$



Just right!

好的拟合

$$\theta_0 + \theta_1 x + \theta_2 x^2$$



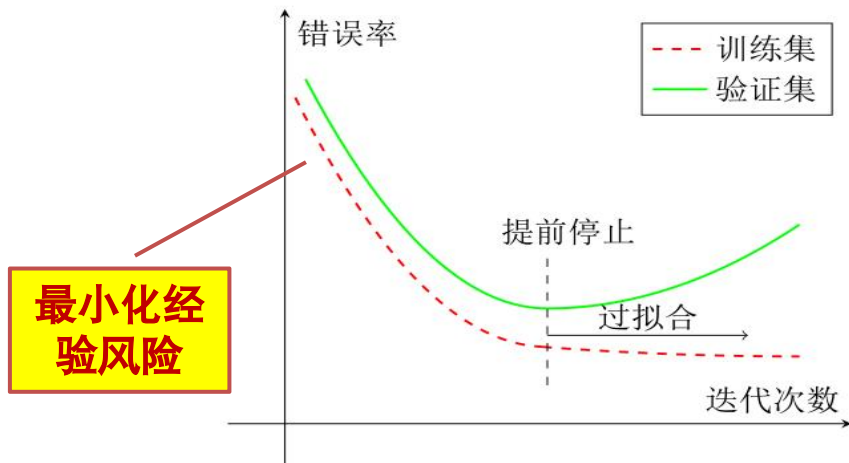
overfitting

过拟合

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

- 表达力好的模型，可以较好地对训练数据中的规律和模式进行学习；
- 复杂度低的模型，方差较小，不容易过拟合，有较好的泛化表达。

- ◆ **过拟合**：所构建的机器学习模型在训练集上表现的很好，但在测试集上表现很差，推广泛化能力差。
- ◆ 过拟合问题往往是由于训练数据少和噪声等原因造成的。
- ◆ **解决方法**：1.数据集扩增；2.对数据进行K折交叉验证；3.选取合适的停止训练标准；4.正则化；5.降维（PCA）等。



- 模型调优:如何选择最优的模型

验证集 评估选择

切分数据为训练集和验证集。

对于候选超参数，在训练集上进行模型训练，在验证集上评估

网格搜索 交叉验证

通过网格搜索/随机搜索产出候选的超参数组。

对参数组的每一组超参数，使用交叉验证评估效果。

选出效果最好的超参数。

- 模型调优: 如何选择最优的模型

```
from sklearn.model_selection import GridSearchCV #导入网格搜索交叉验证
```

```
params = {"C": [0.1,0.5,0.8,1.0, 5.0, 10.0], "kernel": ['linear', 'rbf']}  
clf = svm.SVC(decision_function_shape='ovr')
```

```
grid = GridSearchCV(clf, params, scoring="f1_macro", cv=5)  
grid_result = grid.fit(X_train, y_train) #用训练数据进行网格搜索出模型的最佳超参数  
print("最佳结果", grid_result.best_score_)  
print("最佳参数", grid_result.best_params_)  
print("最佳估计器", grid_result.best_estimator_)
```

最佳结果 0.7821430856724975

最佳参数 {'C': 0.8, 'kernel': 'linear'}

最佳估计器 SVC(C=0.8, kernel='linear')

- 模型调优: 如何选择最优的模型

```
y_pred=grid.predict(X_test)
acc, f1_macro, precision_macro, recall_macro = (
    metrics.accuracy_score(y_test, y_pred),
    metrics.f1_score(y_test, y_pred, average='macro'),
    metrics.precision_score(y_test, y_pred, average='macro'),
    metrics.recall_score(y_test, y_pred, average='macro')
)
# 打印测试结果, 保留四位小数
print(f'调优后测试结果: F1: {f1_macro:.2f}, P: {precision_macro:.2f}, R:
{recall_macro:.2f}, acc: {acc:.2f}')
```

调优后测试结果: F1: 0.89, P: 0.90, R: 0.90, acc: 0.89

调优前测试结果: F1: 0.84, P: 0.88, R: 0.86, acc: 0.84

- 1.假设有1000张猫狗的图片， 每张图片都有标签， 请描述如何用留出法和k折交叉验证划分数据集？
- 2.在机器学习中， 什么是过拟合？ 请给出几种解决过拟合的策略（要求至少给出2种）。
- 3.在机器学习中， 什么是k折交叉验证？ k折交叉验证的作用是什么？

- 例题：有100张照片，其中猫的照片有60张，狗的照片是40张。输入这100张照片进行**二分类**识别，找出这100张照片中的所有的猫。
- 识别结果的**混淆矩阵**如下图所示，图中：正例（Positives）：识别对的；负例（Negatives）：识别错的。
- 要求：计算识别结果的**正确率**(Accuracy)、**精度**(Precision)、**召回率**(Recall)和**F1分数**(F-measure)。



识别结果的混淆矩阵

$$\text{精确率 (Precision)} : Precision = \frac{TP}{TP+FP}$$

$$\text{召回率 (Recall)} : Recall = \frac{TP}{TP+FN}$$

$$\text{F-measure} : F - measure = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$\text{准确率 (Accuracy)} : Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

		预测值	
		Positive	Negative
实际值	Positive	TP=40	FN=20
	Negative	FP=10	TN=30

- 线性回归
- 逻辑回归
- SVM (支持向量机)
- 决策树

监督学习

sklearn:python的重要机器学习库, 其中封装了大量的**机器学习算法**, 如: **分类、回归、降维以及聚类**

numpy: python第三方库, 用于**科学计算**

matplotlib:python第三方库, 主要用于进行**可视化**

- K-means聚类
- 强化学习

- ✓ 不需要标签, 选择的行动越好, 得到的反馈越多。
- ✓ **AlphaGo Zero**用的就是强化学习。

回归问题

线性回归
广义线性回归
岭回归
Lasso回归

分类问题

决策树
贝叶斯
支持向量机
逻辑回归
集成学习

聚类问题

K-means
高斯混合聚类
密度聚类
层次聚类